clvlib: a library to compute covariant Lyapunov vectors

Riccardo Consonni, Luca Magri

April 2026

1 Introduction

`clvlib` is a Python library for computing Lyapunov exponents (LEs) [10], backward Lyapunov vectors (BLVs) (also known as Optimal Time Dependent modes [1]) and covariant Lyapunov vectors [5] (CLVs) of nonlinear dynamical systems. The library is written using NumPy [6], SciPy [12], and PyTorch [9], and its purpose is to package standard algorithms behind a small interface that is easier to reuse, inspect, and extend.

The library uses the following numerical workflow. Lyapunov exponents and BLVs are estimated by integrating the variational equations and repeatedly orthonormalising the tangent basis, following Benettin's method [2]. CLVs are then computed from the stored factors using the Ginelli algorithm [4]. The library also includes utilities to compute angles between vectors and subspaces, and to evaluate instantaneous covariant Lyapunov exponents (ICLEs) [7] [3] along a trajectory.

2 QR Decomposition

A central step in the library is the repeated QR factorisation of the tangent basis matrix. Given an evolved basis Y, the algorithm writes it as $Y = QR$, where Q has orthonormal columns and R is upper triangular. In this setting, the columns of Q define the backward Lyapunov vectors, while the diagonal entries of R contain the local stretching information used to estimate finite-time growth rates and Lyapunov exponents.

`clvlib` uses a sign-corrected QR decomposition: after each factorisation, the signs are chosen so that the diagonal of R is positive. This removes arbitrary sign changes introduced by standard Householder QR routines and keeps the BLV basis consistently oriented in time.

We run 2 tests to compare `clvlib` to other naive implementations. The first test consists in the computation of the CLVs of a Lorenz '63 system, comparing the CLVs obtained from a naive Householder implementation with the sign-corrected version used in `clvlib`. In the left panel of Figure 1, the naive

Householder implementation shows spurious sign flips, while the sign-corrected version shows the correct temporal evolution of the CLVs.

Many bypass this spurious behavior by using Gram–Schmidt (GS) orthogonalization, which preserves the orientation of the basis. Such solution, however, is often computationally more expensive as it is not implemented in high performance computing libraries. Moreover Householder reflections-based QR decompositions are numerically more stable [11].

For this reason we run a second test comparing the computational cost of `clvlib` with a Numba [8] JIT-compiled Gram–Schmidt implementation. The test is run over 100 integration timesteps of a Lorenz '96 system for different number of degrees of freedom, or dimensions. The results are shown in the right panel of Figure 1. The sign-corrected Householder implementation in `clvlib` is comparable in speed to the JIT-compiled Gram–Schmidt for low dimensions, but as the dimensions increase, the performance gap widens significantly, in Table 1 we summarise the perofrmance speedup.

The code was tested on a AMD Epyc 9654 96-Core Processor, running at 2.4GHz with 1 TB of RAM. The performance comparison was run using Python 3.13.9, NumPy 2.4.4, SciPy 1.17.1, and Numba 0.65.0.

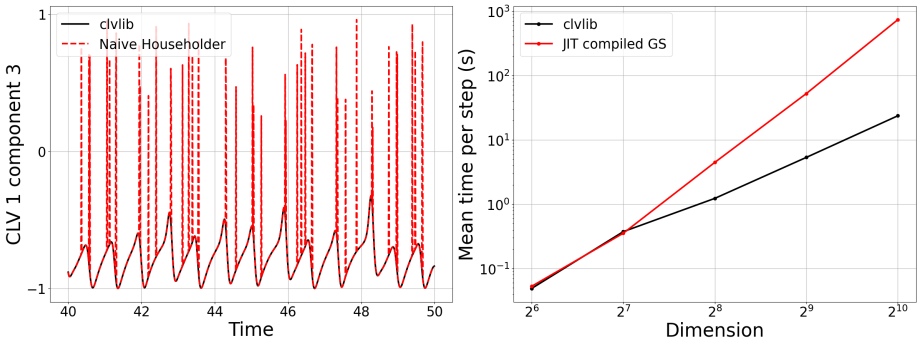

Figure 1: Comparison of the QR step used in `clvlib`. Left panel: Lorenz '63 system CLV 1 third entry. We compare a naive Householder implementation with the sign-corrected version used in `clvlib`, showing the removal of artificial sign flips in the CLVs. Right panel: Computational cost comparison between `clvlib` and a Numba JIT-compiled Gram–Schmidt implementation on a Lorenz '96 system.

3 Software Structure

The main public API is compact. The user supplies a vector field $f(t, x)$, its Jacobian $Df(t, x)$, and either an initial condition or a precomputed trajectory. The functions `lyap_exp` and `lyap_exp_from_ic` return Lyapunov exponents, BLVs (`Q_history`) and their finite time growth rates (`R_history`), while `lyap_analysis` and `lyap_analysis_from_ic` also compute CLVs (`CLV_history`)

Dimension	`clvlib`	Naive GS	Speedup
2^6	0.049 s	0.053 s	$\times$1.08
2^7	0.375 s	0.354 s	$\times$0.944
2^8	1.233 s	4.466 s	$\times$3.62
2^9	5.343 s	52.49 s	$\times$9.81
2^{10}	23.69 s	738.3 s	$\times$31.2

Table 1: Benchmark summary for the Lorenz '96 QR-step comparison.

and their inverse growth rates (`D_history`). This separation is useful because some applications need only the exponents and BLVs, whereas others require the full Lyapunov analysis.

One of the design choices made is the abstraction of the variational stepper. A stepper advances both the state and the tangent basis over one time step. The library provides Euler, RK2, RK4, and discrete-time steppers, but custom steppers can also be registered or passed directly. This makes the package adaptable to different models without changing the rest of the analysis pipeline.

The package also includes post-processing utilities built on top of the CLVs. `compute_angles` evaluates angles between vectors, `principal_angles` computes principal angles between subspaces over time, and `compute_ICLE` computes instantaneous covariant Lyapunov exponents (ICLEs) along a trajectory. As a result, `clvlib` supports both asymptotic and local analyses of instability.

4 Typical Use and Assessment

The tutorials use the Lorenz–63 system as the main example. This reflects the intended workflow: define the governing equations and Jacobian, choose a time grid, call `lyap_analysis_from_ic`, and inspect the resulting trajectory, exponent history, and CLVs. The `n_lyap` option further allows users to compute only the leading unstable directions, which is especially useful when the full tangent basis is not needed.

5 Conclusion

`clvlib` is a lightweight library with a small codebase and a compact API. Its implementation combines the variational stepper interface, which allows different integration schemes to be used without changing the rest of the pipeline, with a sign-corrected QR decomposition that keeps the BLV basis orientation consistent across time steps.

Its contribution is an implementation of established methods that makes CLV computation more accessible in practice. For researchers who want a compact tool for Lyapunov exponents, CLVs, and related geometric diagnostics, it provides a quick starting point.

References

[1] H. Babaee and T. P. Sapsis. A minimization principle for the description of modes associated with finite-time instabilities. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 472(2186):20150779, February 2016.

[2] Giancarlo Benettin, Luigi Galgani, Antonio Giorgilli, and Jean-Marie Strelcyn. Lyapunov Characteristic Exponents for smooth dynamical systems and for hamiltonian systems; a method for computing all of them. Part 1: Theory. *Meccanica*, 15(1):9–20, March 1980.

[3] Riccardo Consonni and Luca Magri. Precursors of extreme events and critical transitions. https://arxiv.org/abs/2604.12869v1, April 2026.

[4] F. Ginelli, P. Poggi, A. Turchi, H. Chaté, R. Livi, and A. Politi. Characterizing Dynamics with Covariant Lyapunov Vectors. *Physical Review Letters*, 99(13):130601, September 2007.

[5] Francesco Ginelli, Hugues Chaté, Roberto Livi, and Antonio Politi. Covariant lyapunov vectors. *Journal of Physics A: Mathematical and Theoretical*, 46(25):254005, 2013.

[6] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[7] Pavel V. Kuptsov and Sergey P. Kuznetsov. Lyapunov analysis of strange pseudohyperbolic attractors: Angles between tangent subspaces, local volume expansion and contraction. *Regular and Chaotic Dynamics*, 23:908–932, 2018.

[8] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: a llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, New York, NY, USA, 2015. Association for Computing Machinery.

[9] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library.* Curran Associates Inc., Red Hook, NY, USA, 2019.

[10] Arkady Pikovsky and Antonio Politi. *Lyapunov Exponents: A Tool to Explore Complex Dynamics*. Cambridge University Press, 2016.

[11] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 1997.

[12] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.